

Name:N.Madhusudhan

Reg no : 192425303

Job Scheduling with Deadlines

Algorithm

1. For each job, store its job ID, deadline, and profit.
2. Sort all the jobs in decreasing order of profit.
3. Find the maximum deadline among all jobs; this gives the number of available time slots (slot 1 to maximum deadline).
4. For each job, taken in sorted order, search for a free time slot starting from its own deadline and moving backwards towards slot 1.
5. If a free slot is found, assign the job to that slot and add its profit to the total profit.
6. If no free slot is available before the job's deadline, reject the job.
7. After all jobs are processed, the filled slots give the optimal job sequence, and the accumulated sum gives the maximum profit.

Pseudocode

```
ALGORITHM JobScheduling(jobs, deadline, profit, n)
  create job records (id, deadline, profit) for i = 1 to n
  sort jobs in decreasing order of profit

  maxDeadline <- max(deadline[1..n])
  create slot[1..maxDeadline], initialize all to EMPTY
  totalProfit <- 0

  for each job in sorted jobs do
    for t <- min(job.deadline, maxDeadline) downto 1 do
      if slot[t] = EMPTY then
        slot[t] <- job.id
        totalProfit <- totalProfit + job.profit
        break
      end if
    end for
  end for

  return totalProfit, slot
```

Program

```
class Job:
    def __init__(self, job_id, deadline, profit):
        self.id = job_id
        self.deadline = deadline
        self.profit = profit

def schedule_jobs(jobs):
    jobs_sorted = sorted(jobs, key=lambda x: x.profit, reverse=True)
    print("=" * 60)
    print("JOB SCHEDULING WITH DEADLINES - GREEDY ALGORITHM")
    print("=" * 60)
    print("\nJOBS SORTED BY PROFIT (DESCENDING)")
    print("-" * 40)
    for j in jobs_sorted:
        print(f"Job {j.id} -> Deadline={j.deadline}, Profit={j.profit}")

    max_deadline = max(j.deadline for j in jobs)
    slots = [None] * (max_deadline + 1)
    total_profit = 0
    scheduled = []

    for j in jobs_sorted:
        print("\n" + "-" * 60)
        print(f"CONSIDER Job {j.id} (Deadline={j.deadline}, Profit={j.profit})")
        print("-" * 60)
        placed = False
        for t in range(min(j.deadline, max_deadline), 0, -1):
            if slots[t] is None:
                slots[t] = j.id
                total_profit += j.profit
                scheduled.append(j.id)
                placed = True
                print(f"Free slot found at time {t} -> Job {j.id} scheduled")
                break
        if not placed:
            print(f"No free slot before deadline -> Job {j.id} rejected")
        print(f"Current schedule: {slots[1:]}")
        print(f"Running total profit: {total_profit}")

    return slots, total_profit, scheduled

def main():
    jobs = [Job(1, 2, 100), Job(2, 1, 19), Job(3, 2, 27), Job(4, 1, 25)]
    print("JOB DETAILS")
```

```

print("-" * 30)
for j in jobs:
    print(f'Job {j.id} -> Deadline={j.deadline}, Profit={j.profit}')

slots, total_profit, scheduled = schedule_jobs(jobs)

print("\n" + "=" * 60)
print("FINAL JOB SEQUENCE")
print("=" * 60)
print("Scheduled slots (time slot -> job):")
for t in range(1, len(slots)):
    print(f'Slot {t} -> {'Job' + str(slots[t]) if slots[t] else 'Empty'}')

print("\n" + "=" * 60)
print("RESULT")
print("=" * 60)
print(f'Jobs scheduled : {[Job' + str(i) for i in scheduled]}')
print(f'Max Profit = {total_profit}')

```

main()

Sample Input

n = 4

deadline = [2, 1, 2, 1]

profit = [100, 19, 27, 25]

Manual Process

Jobs: Job1(deadline=2, profit=100), Job2(deadline=1, profit=19), Job3(deadline=2, profit=27), Job4(deadline=1, profit=25)

Sort by profit (descending): Job1(100), Job3(27), Job4(25), Job2(19)

Maximum deadline = 2 -> 2 time slots available: Slot1, Slot2

Step 1: Consider Job1 (d=2, p=100) -> Slot2 is free -> place Job1 in Slot2. Schedule: [_, Job1]. Total profit = 100

Step 2: Consider Job3 (d=2, p=27) -> Slot2 occupied, check Slot1 -> free -> place Job3 in Slot1. Schedule: [Job3, Job1]. Total profit = 127

Step 3: Consider Job4 (d=1, p=25) -> Slot1 occupied, no earlier slot available -> Job4 rejected.

Step 4: Consider Job2 (d=1, p=19) -> Slot1 occupied, no earlier slot available -> Job2 rejected.

Final schedule: Slot1 -> Job3, Slot2 -> Job1

Max Profit = 100 + 27 = 127

Output

main.py	Output
<pre>1 class Job: 2 def __init__(self, job_id, deadline, profit): 3 self.id = job_id 4 self.deadline = deadline 5 self.profit = profit 6 7 def schedule_jobs(jobs): 8 jobs_sorted = sorted(jobs, key=lambda x: x.profit, 9 reverse=True) 10 max_deadline = max(j.deadline for j in jobs) 11 slots = [None] * (max_deadline + 1) 12 total_profit = 0 13 scheduled = [] 14 for j in jobs_sorted: 15 for t in range(min(j.deadline, max_deadline), 0, -1): 16 if slots[t] is None: 17 slots[t] = j.id 18 total_profit += j.profit 19 scheduled.append(j.id) 20 break 21 return slots, total_profit, scheduled 22 23 def main(): 24 jobs = [Job(1,2,100), Job(2,1,19), 25 Job(3,2,27), Job(4,1,25)] 26 ... 27 28 main()</pre>	<pre>JOB DETAILS ----- Job1 -> Deadline=2, Profit=100 Job2 -> Deadline=1, Profit=19 Job3 -> Deadline=2, Profit=27 Job4 -> Deadline=1, Profit=25 ===== JOB SCHEDULING WITH DEADLINES - GREEDY ALGORITHM ===== JOBS SORTED BY PROFIT (DESCENDING) ----- Job1 -> Deadline=2, Profit=100 Job3 -> Deadline=2, Profit=27 Job4 -> Deadline=1, Profit=25 Job2 -> Deadline=1, Profit=19 ----- CONSIDER Job1 (Deadline=2, Profit=100) ----- Free slot found at time 2 -> Job1 scheduled Running total profit: 100 ----- CONSIDER Job3 (Deadline=2, Profit=27) ----- Free slot found at time 1 -> Job3 scheduled Running total profit: 127 ===== RESULT =====</pre>